SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA0311	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	P. Nandu Vardhan	**Reg. No.:**	192425067
Section / Batch:	B.tech AIML-2024	**Date:**	15-10-2026

Code of Conduct

I P. Nandu Vardhan / 192425067 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:
Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
Password
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)
Mobile Number
Must contain exactly 10 digits.
First digit should be between 6–9.
Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Answer:

Regular Expressions – User Registration Validation System

1. Aim

To develop a Python application using Regular Expressions (Regex) to validate a user's email address, password, and mobile number according to predefined security rules.

2. Implementation Approach

Step 1: Accept User Details

Get the email address, password, and mobile number from the user using Python's `input()` function.

Step 2: Validate Email Address

Use a Regex pattern to check whether the email starts with an alphabet, contains permitted characters before `@`, has an alphabetic domain, and ends with an allowed extension such as `.com`, `.org`, `.edu`, `.net`, or `.in`.

Step 3: Validate Password

Check whether the password contains at least eight characters, one uppercase letter, one lowercase letter, one digit, and one special character from `@, #, $, %, &, !`.

Step 4: Validate Mobile Number

Use a Regex pattern to verify that the mobile number contains exactly 10 digits and begins with a digit between 6 and 9.

Step 5: Display Validation Results

Display `Valid Email` or `Invalid Email`, `Strong Password` or `Weak Password`, and `Valid Mobile Number` or `Invalid Mobile Number`, depending on the validation results.

3. Key Part of the Algorithm

1. Read the email, password, and mobile number.
2. Define suitable Regex patterns for each field.
3. Use `re.fullmatch()` to check the complete email and mobile number.
4. Use lookahead assertions in the password pattern to check all password requirements.
5. Store each validation result as a Boolean value.
6. Display the appropriate validation message for each field.

4. Python Code

```
import re

# Get user details
email = input("Enter Email Address: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

# Email validation
email_pattern = r"[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)"
email_valid = bool(re.fullmatch(email_pattern, email))

# Password validation
password_pattern = (
    r"(?=.*[A-Z])"
    r"(?=.*[a-z])"
    r"(?=.*\d)"
    r"(?=.*[@#$%&!])"
    r".{8,}"
)
password_valid = bool(re.fullmatch(password_pattern, password))

# Mobile number validation
mobile_pattern = r"[6-9]\d{9}"
mobile_valid = bool(re.fullmatch(mobile_pattern, mobile))

# Display results
print("\n--- Validation Results ---")

print("Valid Email" if email_valid else "Invalid Email")
print("Strong Password" if password_valid else "Weak Password")
print("Valid Mobile Number" if mobile_valid
```

```
else "Invalid Mobile Number")
```

5. Sample Input

Enter Email Address: student123@college.edu
Enter Password: Hello@123
Enter Mobile Number: 9876543210

6. Expected Output

--- Validation Results ---
Valid Email
Strong Password
Valid Mobile Number

7. Important Regex Patterns

Pattern	Purpose
`[A-Za-z]`	Matches an alphabet
`\d`	Matches a digit
`(?=.*[A-Z])`	Requires an uppercase letter
`(?=.*[a-z])`	Requires a lowercase letter
`(?=.*[@#$%&!])`	Requires an allowed special character
`.{8,}`	Requires at least 8 characters
`[6-9]\d{9}`	Matches a 10-digit mobile number beginning with 6–9
`re.fullmatch()`	Checks the entire input against the pattern

8. Conclusion

The application uses Python's Regular Expressions to validate email addresses, password strength, and mobile numbers. It applies predefined rules and displays clear validation messages, providing a simple, intermediate-level implementation of input validation for a user registration system.

Note: The email pattern follows the specified simplified rules. It intentionally does not support every possible valid email format, and successful pattern matching does not confirm that an email address or mobile number actually exists.

```
# Mobile number validation
mobile_pattern = r"[6-9]\d{9}"
mobile_valid = bool(re.fullmatch(mobile_pattern, mobile))

# Display results
print("\n--- Validation Results ---")

print("Valid Email" if email_valid else "Invalid Email")
print("Strong Password" if password_valid else "Weak Password")
print("Valid Mobile Number" if mobile_valid
      else "Invalid Mobile Number")
```

```
Enter Email Address:  student123@college.edu
Enter Password:  Hello@123
Enter Mobile Number:  9876543210

--- Validation Results ---
Valid Email
Strong Password
Valid Mobile Number
```

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
  - States
  - Input alphabet
  - Transition table
  - Initial state
  - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output

Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Answer:

Section 2: Design a Finite State Automata (FSA) Simulator

1. Aim

To develop a Python-based Deterministic Finite Automaton (DFA) simulator that accepts a DFA description, processes multiple input strings, simulates state transitions, and determines whether each string is accepted or rejected.

2. Implementation Approach

Step 1: Define the DFA

Specify the set of states, input alphabet, transition table, initial state, and final states.

Step 2: Create the Transition Table

Store the transitions using a Python dictionary. Each state and input symbol determine the next state.

Step 3: Accept Input Strings

Allow the user to enter multiple strings to test against the DFA.

Step 4: Simulate State Transitions

Start from the initial state and process each input character one by one. Use the transition table to determine the next state and record the transition path.

Step 5: Check Acceptance

After processing the entire string, check whether the current state belongs to the set of final states. If it does, the string is accepted; otherwise, it is rejected.

Step 6: Display Results

Display the transition path and the final acceptance status for each input string.

3. Key Part of the Algorithm

1. Initialize the DFA states, alphabet, transition table, initial state, and final states.
2. Read the number of input strings and accept each string.
3. Set the current state to the initial state.
4. For each input symbol, find the next state using the transition table.
5. Record each state in the transition path.
6. Check whether the final state is an accepting state.
7. Display the transition path and the result.
8.

4. Python Code

```
# DFA Simulator for strings ending with "ab"

# DFA description
states = ["q0", "q1", "q2"]
alphabet = ["a", "b"]
initial_state = "q0"
final_states = ["q2"]

# Transition table
transitions = {
    "q0": {"a": "q1", "b": "q0"},
    "q1": {"a": "q1", "b": "q2"},
    "q2": {"a": "q1", "b": "q0"}
}

# Function to simulate DFA
def simulate_dfa(string):
    current_state = initial_state
    path = [current_state]

    for symbol in string:
        if symbol not in alphabet:
            print("Invalid input symbol:", symbol)
            return

        current_state = transitions[current_state][symbol]
        path.append(current_state)

    print("Input String:", string)
    print("Transition Path:", " -> ".join(path))

    if current_state in final_states:
        print("Result: Accepted")
    else:
        print("Result: Rejected")

    print()


# Accept multiple input strings
n = int(input("Enter number of strings: "))

for i in range(n):
    string = input("Enter input string: ")
    simulate_dfa(string)
```

5. Sample Input

```
Enter number of strings: 3
Enter input string: abaab
Enter input string: aabb
Enter input string: ab
```

6. Expected Output

```
Input String: abaab
Transition Path: q0 -> q1 -> q1 -> q2 -> q1 -> q2
Result: Accepted
```

Input String: aabb
Transition Path: q0 -> q1 -> q1 -> q2 -> q0
Result: Rejected

Input String: ab
Transition Path: q0 -> q1 -> q2
Result: Accepted

7. Conclusion

The DFA simulator demonstrates how a Deterministic Finite Automaton processes input strings using a transition table. It tracks state transitions and determines acceptance based on the final state. The implementation uses Python dictionaries, loops, lists, and conditional statements to simulate a finite automaton in a simple and understandable way.

```
    print("Input String:", string)
    print("Transition Path:", " -> ".join(path))

    if current_state in final_states:
        print("Result: Accepted")
    else:
        print("Result: Rejected")

    print()


# Accept multiple input strings
n = int(input("Enter number of strings: "))

for i in range(n):
    string = input("Enter input string: ")
    simulate_dfa(string)
```

```
Enter number of strings:  3
Enter input string:  abaab
Input String: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Result: Accepted

Enter input string:  aabb
Input String: aabb
Transition Path: q0 -> q1 -> q1 -> q2 -> q0
Result: Rejected

Enter input string:  ab
Input String: ab
Transition Path: q0 -> q1 -> q2
Result: Accepted
```

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix

Expected Output
Display all matching patterns based on user selection.

Section 3: Smart Pattern Matching Engine using Regular Expressions

1. Aim

To develop a Python-based text search engine using Regular Expressions (Regex) to identify words, prefixes, suffixes, dates, phone numbers, hashtags, and mentions from a given text.

2. Implementation Approach

Step 1: Input Text

Accept a paragraph or a collection of text from the user. This text acts as the source for pattern matching.

Step 2: Define Regex Patterns

Create suitable regular expression patterns for dates, phone numbers, hashtags, mentions, prefixes, and suffixes.

Step 3: Select Search Operation

Display a menu that allows the user to choose the required search operation.

Step 4: Perform Pattern Matching

Use Python's `re.findall()` function to find all matching patterns in the text. Use `re.escape()` when including a user-provided keyword in a pattern.

Step 5: Display Results

Display all matching items for the selected operation. If no matches are found, display an appropriate message.

Step 6: Repeat Searches

Allow the user to perform multiple searches on the same text until they choose to exit.

3. Key Part of the Algorithm

1. Read the input text.
2. Define Regex patterns for dates, phone numbers, hashtags, mentions, words, prefixes, and suffixes.
3. Display a menu containing the available search operations.
4. Accept the user's choice and any required search keyword.
5. Apply the corresponding Regex pattern using `re.findall()`.
6. Display all matching patterns or a message indicating that no matches were found.
7. Repeat the process until the user selects Exit.

4. Python Code

```
import re

# Input text
text = input("Enter text to search: ")

# Function to perform pattern matching
def search_pattern(choice):

    if choice == 1:
        # Search dates in DD/MM/YYYY format
        pattern = r"\b\d{2}/\d{2}/\d{4}\b"

    elif choice == 2:
        # Search 10-digit mobile numbers
        pattern = r"\b[6-9]\d{9}\b"

    elif choice == 3:
        # Search hashtags
        pattern = r"(?<!\w)#[A-Za-z0-9_]+"

    elif choice == 4:
        # Search @username mentions
        pattern = r"(?<!\w)@[A-Za-z0-9_]+"

    elif choice == 5:
        # Prefix search
        prefix = input("Enter prefix: ")
        pattern = r"\b" + re.escape(prefix) + r"\w*"

    elif choice == 6:
        # Suffix search
        suffix = input("Enter suffix: ")
        pattern = r"\b\w*" + re.escape(suffix) + r"\b"

    elif choice == 7:
        # Word search
```

```
        word = input("Enter word: ")
        pattern = r"\b" + re.escape(word) + r"\b"

    else:
        print("Invalid choice!")
        return

    # Find matching patterns
    matches = re.findall(pattern, text, re.IGNORECASE)

    # Display results
    if matches:
        print("Matching Patterns:")
        for item in matches:
            print(item)
        print("Total Matches:", len(matches))
    else:
        print("No matching patterns found.")


# Menu
while True:
    print("\n--- Smart Pattern Matching Engine ---")
    print("1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Search Word")
    print("0. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 0:
        print("Search engine closed.")
        break

    search_pattern(choice)
```

5. Sample Input

Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

Enter the complete text as a single line when prompted, or modify the input statement to accept multiple lines.

For example:

Meeting on 12/09/2026 Call 9876543210 #NLP @OpenAI natural language processing

6. Sample Output

For menu option `1` (Search Date):

Matching Patterns:
12/09/2026

Total Matches: 1

For menu option `3` (Search Hashtag):

Matching Patterns:
#NLP
Total Matches: 1

For menu option `4` (Search Mention):

Matching Patterns:
@OpenAI
Total Matches: 1

7. Important Regex Patterns

Pattern	Purpose
`\d{2}/\d{2}/\d{4}`	Matches dates in DD/MM/YYYY format
`[6-9]\d{9}`	Matches 10-digit Indian mobile numbers
`#[A-Za-z0-9_]+`	Matches hashtags
`@[A-Za-z0-9_]+`	Matches username mentions
`\b`	Identifies word boundaries
`re.findall()`	Returns all matching patterns

8. Conclusion

The Smart Pattern Matching Engine uses Python's Regular Expressions to search text for different patterns, including dates, mobile numbers, hashtags, mentions, prefixes, suffixes, and specific words. The menu-driven implementation demonstrates practical text processing using Regex, functions, loops, and conditional statements.

Note: The date pattern checks the format, not whether a date exists on the calendar. Similarly, matching a phone number's format does not verify that the number is active.

```
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Search Word")
    print("0. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 0:
        print("Search engine closed.")
        break

    search_pattern(choice)
```

```
Enter text to search:  Meeting on 12/09/2026 Call 9876543210 #NLP @OpenAI natural language processing

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
Enter your choice:  2
Matching Patterns:
9876543210
Total Matches: 1

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
Enter your choice:  5
Enter prefix:  ↑↓ for history. Search history with c-↑/c-↓
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____